

# 05 — Standard Library & Forensics Functions

---

## Overview

JOCKY's standard library provides 15 built-in functions for forensic analysis. These functions are available in every JOCKY program without any import or declaration.

The same function name is implemented in two places:

- `jocky/stdlib.py` — Python ctypes callbacks, used when the compiler runs in `--run` (JIT) mode
- `stdlib/forensics.c` — C implementations, compiled by gcc and linked into native `.exe` files

Both implementations have identical signatures. From JOCKY's perspective (and the compiler's), there is no difference — both are just external C functions.

---

## How Stdlib Is Wired Up

### In JIT mode

Before LLVM's JIT engine links the compiled code, the compiler calls `register_all()` from `jocky/stdlib.py`:

```
def register_all() -> None:
    builders = {
        'report': _cb_report,
        'procs_list': _cb_procs_list,
        ...
    }
    for name, builder_fn in builders.items():
        cb = builder_fn() # create the ctypes callback object
        addr = ctypes.cast(cb, ctypes.c_void_p).value # get its address
        llvm.add_symbol(name, addr) # register with LLVM's symbol table
        _CALLBACKS[name] = cb # prevent garbage collection
```

`llvm.add_symbol('report', addr)` tells LLVM: "when the compiled code jumps to the external symbol `report`, jump to this memory address instead." The address is the entry point of the ctypes callback, which is a C-compatible function pointer wrapping the Python function.

### In native binary mode

When the compiled JOCKY `.o` file contains a call to `report`, it emits:

```
call void @report(i8* ...)
```

with the address left as a placeholder relocation. The linker resolves this by looking up `report` in `forensics.o`'s symbol table. `gcc` handles this automatically:

```
gcc hello.o forensics.o _jocky_entry.o -o hello.exe
```

## ctypes Callbacks — How Python Functions Become C Functions

`ctypes.CFUNCTYPE(return_type, *arg_types)` creates a C-compatible function type descriptor. When used as a decorator, it wraps a Python function in a C-callable wrapper:

```
@ctypes.CFUNCTYPE(None, ctypes.c_char_p)
def report(msg: bytes) -> None:
    if msg:
        print(f"[JOCKY] {msg.decode('utf-8', errors='replace')}")
```

- `None` as the first argument → C `void` return
- `ctypes.c_char_p` → C `const char*` parameter; Python receives it as `bytes`

The decorated `report` is now a `ctypes` function object. Its address (`ctypes.cast(report, ctypes.c_void_p).value`) is a raw memory pointer to C code that LLVM's JIT can call directly.

**Critical: preventing garbage collection.** `ctypes` function objects are Python objects. Python's garbage collector will free them when no Python variable holds a reference. If the callback is freed while JIT code might still call it, the program crashes. The `_CALLBACKS` dict in `stdlib.py` holds a reference to every callback for the duration of the JIT execution:

```
_CALLBACKS: dict[str, object] = {}
# ...
_CALLBACKS[name] = cb    # keeps cb alive
```

## Type Mapping: JOCKY → C → ctypes

| JOCKY type | C type      | ctypes type                                             |
|------------|-------------|---------------------------------------------------------|
| num        | int64_t     | ctypes.c_int64                                          |
| dec        | double      | ctypes.c_double                                         |
| text       | const char* | ctypes.c_char_p (receives as <code>bytes</code> )       |
| flag       | int64_t     | ctypes.c_int64 (0 or 1)                                 |
| raw        | void*       | ctypes.c_void_p (receives/returns as <code>int</code> ) |
| nothing    | void        | None (as return type in CFUNCTYPE)                      |

**Why `raw` maps to `c_void_p` → `int`:** ctypes represents `void*` as a Python `int` holding the raw memory address. You cannot dereference it in Python. This is intentional — `raw` values are opaque handles passed between stdlib functions.

---

## All 14 Standard Library Functions

---

`report(message : text) -> nothing`

**Purpose:** Output a message to stdout.

**C signature:** `void report(const char* message)`

**JIT Python:**

```
@ctypes.CFUNCTYPE(None, ctypes.c_char_p)
def report(msg: bytes) -> None:
    if msg:
        print(f"[JOCKY] {msg.decode('utf-8', errors='replace')}")
```

**Native C:**

```
void report(const char* message) {
    if (message) {
        printf("[JOCKY] %s\n", message);
        fflush(stdout);
    }
}
```

**Real Windows API replacement:**

```
WriteConsoleA(GetStdHandle(STD_OUTPUT_HANDLE), message, strlen(message), NULL,
NULL);
```

**Example:**

```
report(`Scanning started`)
report(`Process found`)
```

---

`procs_list() -> raw`

**Purpose:** Take a snapshot of the currently running processes. Returns an opaque handle that is passed to the other `proc_*` functions.

### JIT Python:

```
@ctypes.CFUNCTYPE(ctypes.c_void_p)
def procs_list() -> int:
    return 1 # non-null sentinel; Python code uses its own _PROCESS_TABLE
```

The Python implementation returns a dummy value (1) because the actual process data is stored in `_PROCESS_TABLE` in Python memory. The other `proc_*` callbacks ignore the handle value and use `_PROCESS_TABLE` directly.

### Native C (stub):

```
void* procs_list(void) {
    return (void*)&g_procs; // pointer to internal struct
}
```

### Real Windows API:

```
// Requires -lpsapi
DWORD pids[1024]; DWORD cbNeeded;
EnumProcesses(pids, sizeof(pids), &cbNeeded);
int count = cbNeeded / sizeof(DWORD);
ProcessList* pl = malloc(sizeof(ProcessList));
pl->count = 0;
for (int i = 0; i < count; i++) {
    HANDLE h = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ, FALSE,
pids[i]);
    if (h) {
        HMODULE hMod; DWORD cbMod;
        if (EnumProcessModules(h, &hMod, sizeof(hMod), &cbMod))
            GetModuleBaseNameA(h, hMod, pl->names[pl->count], 260);
        pl->pids[pl->count++] = pids[i];
        CloseHandle(h);
    }
}
return pl;
```

---

```
proc_count(procs : raw) -> num
```

**Purpose:** Get the number of processes in the snapshot.

### Usage:

```
var procs : raw := procs_list()
var n      : num := proc_count(procs)
```

### JIT Python:

```
@ctypes.CFUNCTYPE(ctypes.c_int64, ctypes.c_void_p)
def proc_count(procs: int) -> int:
    return len(_PROCESS_TABLE)
```

`proc_name(procs : raw, index : num) -> text`

**Purpose:** Get the name of the process at `index` (0-based) in the snapshot.

**Critical implementation detail (JIT):** This function must return a pointer to a stable C string. The JIT's LLVM-compiled code will receive an `i8*` pointer and dereference it. If the pointer is invalid (points to freed memory), the program crashes.

The solution: pre-allocate `ctypes.create_string_buffer` objects for each process name and store them in `_NAME_BUFS` (a module-level list). These buffers are never freed because they are always referenced.

```
_NAME_BUFS: list[ctypes.Array] = [
    ctypes.create_string_buffer(name.encode('utf-8') + b'\x00')
    for name, _ in _PROCESS_TABLE
]

@ctypes.CFUNCTYPE(ctypes.c_void_p, ctypes.c_void_p, ctypes.c_int64)
def proc_name(procs: int, idx: int) -> int:
    if 0 <= idx < len(_NAME_BUFS):
        return ctypes.cast(_NAME_BUFS[idx], ctypes.c_void_p).value
    return 0
```

`ctypes.cast(buffer, c_void_p).value` extracts the raw memory address of the buffer as an integer. LLVM JIT code receives this integer as an `i8*` and can read the bytes at that address.

### Usage:

```
var name : text := proc_name(procs, 0)
check (name is `svchost.exe`) {
    report(`found it`)
}
```

`proc_pid(procs : raw, index : num) -> num`

**Purpose:** Get the PID (Process ID) of the process at `index`.

```
var pid : num := proc_pid(procs, 0)    ## PID of first process
```

**Real Windows API (after real `procs_list`):** The PID is already stored in the `ProcessList.pids` array alongside the name.

---

`proc_kill(pid : num) -> nothing`

**Purpose:** Terminate a process by its PID.

**Stub behaviour:** Prints what it would do (safe for demo).

**Real Windows API:**

```
HANDLE h = OpenProcess(PROCESS_TERMINATE, FALSE, (DWORD)pid);
if (h) {
    TerminateProcess(h, 1);
    CloseHandle(h);
}
```

**Usage:**

```
var pid : num := proc_pid(procs, i)
proc_kill(pid)
```

---

`proc_mem_read(pid : num, addr : num, size : num) -> raw`

**Purpose:** Read `size` bytes from process `pid`'s memory at virtual address `addr`.

**Stub behaviour:** Prints the call, returns a zeroed buffer.

**Real Windows API:**

```
HANDLE h = OpenProcess(PROCESS_VM_READ, FALSE, (DWORD)pid);
void* buf = malloc((size_t)size);
SIZE_T nRead;
ReadProcessMemory(h, (LPCVOID)(uintptr_t)addr, buf, (SIZE_T)size, &nRead);
CloseHandle(h);
return buf;
```

---

`net_conns() -> raw`

**Purpose:** Get a list of active TCP/UDP network connections.

**Stub behaviour:** Prints and returns a dummy handle.

**Real Windows API** (requires `-liphlpapi`):

```
MIB_TCPTABLE2* table = NULL; DWORD size = 0;
GetTcpTable2(NULL, &size, TRUE);
table = malloc(size);
GetTcpTable2(table, &size, TRUE);
return table;
```

---

`net_sniff(duration_ms : num) -> raw`

**Purpose:** Capture raw network packets for `duration_ms` milliseconds.

**Stub:** Prints and returns dummy handle.

**Real implementation:** Would use raw sockets (`socket(AF_INET, SOCK_RAW, IPPROTO_IP)`) or WinPcap/npcap.

---

`reg_read(key : text, value_name : text) -> text`

**Purpose:** Read a value from the Windows registry.

**Example:**

```
var val : text := reg_read(`SOFTWARE\Microsoft\Windows\CurrentVersion`,
`ProgramFilesDir`)
```

**Real Windows API** (requires `-ladvapi32`):

```
HKEY hKey; char buf[1024]; DWORD bufLen = sizeof(buf);
RegOpenKeyExA(HKEY_LOCAL_MACHINE, key, 0, KEY_READ, &hKey);
RegQueryValueExA(hKey, value_name, NULL, NULL, (LPBYTE)buf, &bufLen);
RegCloseKey(hKey);
return _strdup(buf);
```

---

`reg_list(key : text) -> raw`

**Purpose:** List subkeys under a registry key.

**Real Windows API:** `RegEnumKeyExA` in a loop.

---

`file_list(path : text) -> raw`

**Purpose:** List files in a directory.

**Real Windows API:** `FindFirstFileA` / `FindNextFileA` / `FindClose`.

---

`file_read(path : text) -> raw`

**Purpose:** Read the binary contents of a file.

**Real Windows API:** `CreateFileA` + `ReadFile` + `CloseHandle`.

---

`sys_info() -> raw`

**Purpose:** Get system information (CPU count, OS version, etc.).

**Real Windows API:** `GetSystemInfo()`, `GetVersionExA()`.

---

`hash_file(path : text) -> raw`

**Purpose:** Compute the SHA-256 hash of a file. Returns 32 bytes.

**Real Windows API** (requires `-lbcrypt`):

```
BCRYPT_ALG_HANDLE hAlg;
BCryptOpenAlgorithmProvider(&hAlg, BCRYPT_SHA256_ALGORITHM, NULL, 0);
// ... open file, hash in chunks ...
// result: 32-byte array
BCryptCloseAlgorithmProvider(hAlg, 0);
```

---

## How to Replace Stubs with Real Implementations

1. Open `stdlib/forensics.c`
2. Find the function you want to make real
3. Replace the stub body with real Windows API code
4. Add the required linker flag to the link command in `compiler.py`:

```
link_cmd = [
    GCC,
    obj_path,
    forensics_o,
    entry_o,
    '-o', exe_path,
    '-mconsole',
    '-lpsapi',      # for proc_* functions
    '-liphlpapi',  # for net_conns
```

```
'-ladvapi32', # for reg_read, reg_list  
'-lbcrypt',   # for hash_file  
]
```

5. Run `python build_stdlib.py` to recompile `forensics.o`

6. Recompile your JOCKY program

---

## The `_PROCESS_TABLE` — Demo Data

In JIT mode, the process list is hardcoded:

```
_PROCESS_TABLE = [  
    ('svchost.exe', 1234),  
    ('explorer.exe', 5678),  
    ('lsass.exe', 9012),  
    ('winlogon.exe', 3456),  
    ('csrss.exe', 7890),  
    ('cmd.exe', 2222),  
    ('python.exe', 3333),  
]
```

This ensures the forensics demo always finds `svchost.exe` and `explorer.exe` (showing detection works) and never finds `malware.exe` (showing the clean path works).

To add processes to the demo, add entries to this list. To test detection, add `('malware.exe', 9999)` and watch the scanner alert.